

CODEXSUN Documentation Portal

Screen by screen Word clone of the Docs reader, Ideas workspace, document editor, navigation, and helper experience.

 Documentation portal overview  Search  Settings

This Word document is a screen by screen visual clone of the current Docs portal. It recreates the persistent navigation rail, focused content surface, document header actions, Ideas pages, document helper, and rich text editing view for review outside the browser.

INCLUDED PORTAL VIEWS

Reader	Document header, source path, edit share copy actions, article body, and helper rail.
Ideas	Development plan and ownership principles shown as a navigable workspace.
Editor	Dedicated editing page using the shared rich-text editor pattern.
Navigation	Repository ownership groups and nested document discovery.
Helper	Contextual outline and quick editorial access.

Clone scope Each page recreates the Docs layout, labels, navigation, and visible actions. It is a Word snapshot rather than an interactive application.

DOCS

[Overview](#)[Ideas](#)[Assists](#)[Platform](#)[Docs](#)[Packages](#)[Repository](#)

Module architecture

[assist/architecture/module-architecture.md](#)

Architecture model

CODEXSUN is a modular monolith. Each business capability has one owner module. Applications compose public packages and contracts without importing another application's private source.

Composition model

[provider](#) → [module](#) → [add on](#) → [app](#) → [platform](#) → [deployable](#)

The lower layers do not import higher layers. A module owns its behavior, data access, public route surface, migrations, events, and tests.

DOCUMENT HELPER

[On this page](#)[Architecture model](#)[Composition model](#)[Open in editor](#)

Reader view The header keeps source actions together, the navigation preserves repository ownership, and the helper stays visible while reading.

DOCS

[Overview](#)[Ideas](#)[Development plan](#)[Architecture standards](#)[Shared packages](#)[Assists](#)

Ideas

What we are developing

CODEXSUN is a modular application platform. The framework remains runtime neutral; Platform Core supplies technical contracts; applications choose infrastructure; business modules own their workflows.

Development flow

1 Real consumer A named product need	2 Owned contract Stable and business free	3 Application proof Composition and failure flow	4 Verified release Tests docs and lifecycle
--	--	--	--

Ownership pattern

Framework	Manifests validation composition plans lifecycle ordering and diagnostics.
Platform Core	Reusable HTTP request context health shutdown and technical adapter contracts.
Application and module	Concrete infrastructure selected modules migrations data routes and domain workflows.

Ideas page A single route presents decisions, development stages, and ownership boundaries without turning planning into a separate product shell.

Global architecture standards

A multi-developer repository stays understandable when every change has an owner, contract, boundary, and proof. These rules guide a team without centralizing product work.

THE CHANGE PATH

1 Need Describe the product need.	2 Owner Choose the owning boundary.	3 Contract Define public behavior.	4 Change Implement in the owner.	5 Proof Check review and record.
---	---	--	--	--

TEAM STANDARDS

Name one owner Each application and module owns its domain rules routes data and records.	Protect boundaries Share stable contracts and public package exports not private source files.
Control dependencies Validate input at the API edge and keep dependency direction toward the domain.	Prove each change Record the decision run focused checks and review the changed boundary.

CODEXSUN COMPARED WITH THE STANDARD

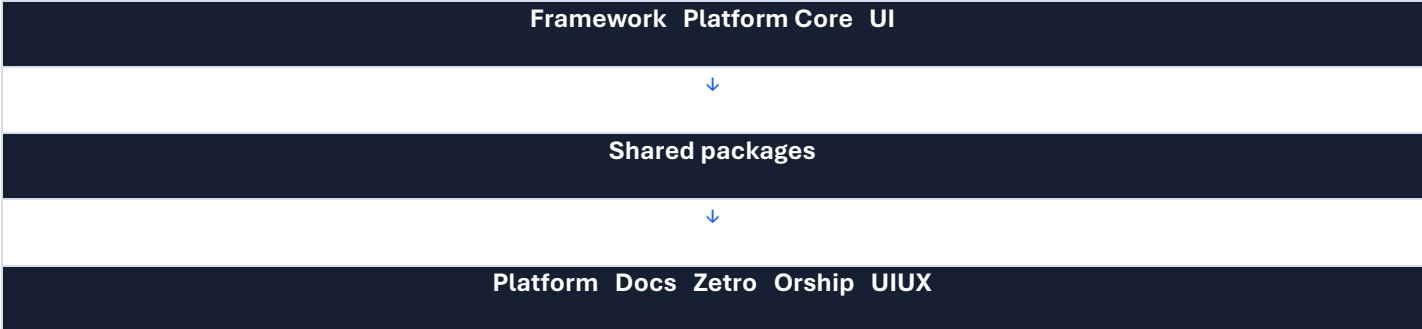
Global standard	Current repository pattern
Source ownership	Apps own business behavior. Packages own reusable technical behavior.
Public contracts	Sibling modules use public index exports and versioned HTTP contracts.
Contributor guidance	AGENTS Assist standards module READMEs and records guide each change.
Integration and release	Workspaces and the runtime catalog build selected applications together.

Working agreement Read the owner guidance, change one boundary, validate it, and record the decision. Shared code grows only after a real application proves the need.

Shared packages and application ownership

CODEXSUN keeps reusable technical capabilities in shared packages and product behavior in source-owned applications. One repository gives teams a common foundation without merging every application into one product.

HOW THE REPOSITORY FITS TOGETHER



SHARED UI VERSUS APPLICATION UI

Shared UI owns Tokens and theme Buttons dialogs tables and form frames MDI layout templates hooks and accessibility behavior	Applications own Business pages domain forms routes and view state API calls validation persistence and workflows Application-specific content and user decisions
--	---

BEFORE A CROSS APPLICATION CHANGE

✓ Choose the owning application or shared package.
✓ Define the public contract before importing or integrating.
✓ Run the owner checks and affected application build.
✓ Update the owner README and development record.

Edit documentation

The editor opens as a dedicated page so writing has a calm full-width surface while source navigation remains available in the browser document.

TITLE

Module architecture

EDITING TOOLBAR

B **I** **S** **</>** **Heading ▼** **•List** **1. List** **Quote** **Link** **Image** **Undo** **Redo** **Write Markdown** **HTML** **Preview**

RICH TEXT EDITOR

Architecture model

CODEXSUN is a modular monolith. Each business capability has one owner module. Applications consume public contracts and reusable UI components.

Composition model

provider → module → add on → app → platform → deployable

The editor keeps a Markdown view and HTML preview available while the canonical saved source remains readable.

Edit action The file-pencil control in the document header opens this page. The rich-text editor provides inline formatting, Markdown and preview modes, link and image entry, and a visible draft status.

Documentation portal structure

This page records the information architecture reproduced in the portal document. It gives implementation and review teams a compact reference for every screen area.

Portal region	Purpose	Key behavior
Top header	Identifies the active document and source path.	Edit, share, and copy source path actions.
Side menu	Navigates Overview, Ideas, Assists, applications, packages, and repository material.	Nested groups follow repository ownership.
Reader	Presents rendered documentation with related references.	Deep links, document outline, and linked notes.
Ideas	Makes product direction and technical standards reviewable.	Development plan, architecture standards, and shared ownership.
Editor	Provides a dedicated writing surface for Docs sources.	Shared rich-text editor with Markdown and preview views.
Document helper	Keeps contextual navigation close to the article.	Outline, source context, and quick open-editor action.

Implementation notes

The Docs web application composes public UI layouts and components. Document discovery, index state, and editing routes remain Docs-owned behavior. The sidebar, header controls, rich-text editor, status area, and responsive accessibility behavior are shared UI concerns.

This Word screen recreates the Docs portal experience for review and sharing. Its controls are represented as visible screen elements; the live Docs application remains the interactive source.